

Assignment 4

ECR setup

Retrieving the ECR repository URI for the hello-service image

```
((base) srimansiramesh@Srimansis-Laptop ~ % ECR_URL=$(aws ecr describe-repositories \
--repository-names hello-service \
--region us-east-1 \
--query 'repositories[0].repositoryUri' \
[ --output text)

[ ((base) srimansiramesh@Srimansis-Laptop ~ % echo "ECR URL: $ECR_URL"

ECR URL: 094522691730.dkr.ecr.us-east-1.amazonaws.com/hello-service
```

Authenticating Docker to ECR

Docker successfully authenticated to ECR via AWS CLI login.

```
((base) srimansiramesh@Srimansis-Laptop ~ % ECR_BASE=$(echo $ECR_URL | cut -d '/' ]
-f1)

((base) srimansiramesh@Srimansis-Laptop ~ % aws ecr get-login-password --region u]
s-east-1 | docker login --username AWS --password-stdin $ECR_BASE
Login Succeeded
```

Verifying upload of docker image to ECR

Docker image pushed to ECR - tagged as latest, status Active

Image Index

Archive

Details

Image tags

latest

URI

094522691730.dkr.ecr.us-east-1.amazonaws.com/hello-service:latest

Digest

sha256:e5e0f824f639ea4dc7418547da01b220fb2b9a0f94a10eb9bf2afc0758991ee0

General information

Artifact type Image Index	Repository hello-service	Pushed at 06 February 2026, 18:09:55 (UTC-05)
Last recorded pull time 06 February 2026, 18:22:03 (UTC-05)	Size (MB) 206.32	Image status Active

Scanning and vulnerabilities

Status

Scan not found

```
(base) srinansiramesh@Srimansis-Laptop part3 % aws ecr list-images \
--repository-name hello-service \
--region us-east-1 \
--query 'imageIds[*].imageTag' \
--output table
|ListImages|
+-----+
| latest |
+-----+
```

ECS cluster

ECS cluster `hello-cluster` active with 1 Fargate task running

> [Clusters](#) > [hello-cluster](#) > Tasks

hello-cluster

Last updated
6 February 2026, 18:25 (UTC-5:00)

Actions

Create with Express Mode

Cluster overview

ARN
arn:aws:ecs:us-east-1:094522691
730:cluster/hello-cluster

Status
Active

CloudWatch monitoring
Default

Registered container instances
-

Services

Draining
-

Active
-

Tasks

Pending
-

Running
1

Services

Tasks

Infrastructure

Metrics

Scheduled tasks

Configuration

Event history

Tags

Tasks (1)

Last updated
6 February 2026, 18:25 (UTC-5:00)

Manage tags

Stop

Run new task

Filter tasks by property or value

Filter desired status
Any desired status

Filter launch type
Any launch type

< 1 >

☐

Task

Last status

Desired status

Task definition

Health status

Created at

☐

[f3aa80b7de444d5da330e...](#)

Running

Running

[hello-task:1](#)

Unknown

3 minutes ago

Task details

Task running on Fargate

> [Clusters](#) > [hello-cluster](#) > [Tasks](#) > [f3aa80b7de444d5da330e26880c8be40](#) > Configuration

f3aa80b7de444d5da330e26880c8be40

Last updated
6 February 2026, 18:25 (UTC-5:00)

Stop

Configuration

Metrics

Logs

Networking

Volumes (0)

Tags

Task overview

ARN
arn:aws:ecs:us-east-1:094522691
730:task/hello-cluster/f3aa80b7de4
44d5da330e26880c8be40

Last status
Running

Desired status
Running

Started/created at
6 February 2026, 18:22 (UTC-5:00)
6 February 2026, 18:21 (UTC-5:00)

Fargate ephemeral storage

Encryption
Default AWS Fargate encryption

Size (GiB)
20

Configuration

Container details for hello-container

< Details

Log configuration

Restart policy

Network bindings

Docker labels and hosts

Environment variables and files

Security Groups

Inbound rules allowing TCP traffic on port 8080

sg-03495d8029f88eae9 - hello-service-sg

Actions

Details

Security group name

hello-service-sg

Security group ID

sg-03495d8029f88eae9

Description

Created in ECS Console

VPC ID

vpc-05b6f59630c3b4e68

Owner

094522691730

Inbound rules count

2 Permission entries

Outbound rules count

1 Permission entry

Inbound rules

Outbound rules

Sharing

VPC associations

Related resources - new

Tags

Inbound rules (2)

Manage tags

Edit inbound rules

Search

	Name	Security group rule ID	IP version	Type	Protocol	Port range
☐	-	sgr-0f43d5101d5a940a4	IPv6	Custom TCP	TCP	8080
☐	-	sgr-0c9378b1a6451cb55	IPv4	Custom TCP	TCP	8080

Network interface

ENI assigned with public IP for external access.

Network interfaces > eni-05d40d5c8af8800a8

Network interface summary for eni-05d40d5c8af8800a8

Delete network interface

Actions

Network interface details

Network interface ID

eni-05d40d5c8af8800a8

Network interface status

In-use

VPC ID

vpc-05b6f59630c3b4e68

Owner

094522691730

Source/dest. check

True

IP addresses

Private IPv4 address

172.31.51.186

IPv6 addresses

-

Association ID

-

IPv4 Prefix Delegation

-

Interface type

Elastic network interface

Subnet ID

subnet-056e342152fb8236b

Requester ID

578734482556

Managed

False

Elastic Fabric Adapter

False

Secondary public IPv4 addresses

-

Elastic IP address owner

amazon

IPv6 Prefix Delegation

-

Description

arn:aws:ecs:us-east-1:094522691730:attachment/a6ef157d-256c-4815-9021-bf575068e06b

Security groups

sg-03495d8029f88eae9 (hello-service-sg)

Availability Zone

us-east-1e

Requester-managed

True

Operator

-

Public IPv4 address

54.236.199.89

Secondary private IPv4 addresses

-

MAC address

06:23:b3:05:82:07

Primary IPv6 address

-

Instance details

Instance ID

-

Instance owner

-

Device index

-

Output

Successfully tested the service endpoints via curl — `/albums` and `/albums/1` both return valid JSON.

```
((base) sriramesh@Sriramesh-Laptop part3 % curl http://54.236.199.89:8080/albums
[
  {
    "id": "1",
    "title": "Blue Train",
    "artist": "John Coltrane",
    "price": 56.99
  },
  {
    "id": "2",
    "title": "Jeru",
    "artist": "Gerry Mulligan",
    "price": 17.99
  },
  {
    "id": "3",
    "title": "Sarah Vaughan and Clifford Brown",
    "artist": "Sarah Vaughan",
    "price": 39.99
  }
]
((base) sriramesh@Sriramesh-Laptop part3 % curl http://54.236.199.89:8080/albums/1
{
  "id": "1",
  "title": "Blue Train",
  "artist": "John Coltrane",
  "price": 56.99
}
```

Map-Reduce

S3 bucket:

S3 bucket `local-map-reducer` with folders for each MapReduce stage: benchmark, chunks, input, mapped, and results.

local-map-reducer Info

Objects

Metadata

Properties

Permissions

Metrics

Management

Access Po

Objects (5)



Copy S3 URI

Copy URL

Download

Open ↗

Delete

Actions ▼

Create folder

Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

< 1 >

☐	Name ▲	Type ▼	Last modified ▼	Size ▼	Storage class
☐	benchmark/	Folder	-	-	-
☐	chunks/	Folder	-	-	-
☐	input/	Folder	-	-	-
☐	mapped/	Folder	-	-	-
☐	results/	Folder	-	-	-

Input files

Two input text files uploaded - `local-sample.txt` (3.7 KB) for initial testing and `sample.txt` (159.1 KB)

Copy S3 URI

Objects

Properties

Objects (2)

Refresh

Copy S3 URI

Copy URL

Download

Open

Delete

Actions

Create folder

Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

< 1 >

Settings

☐	Name	Type	Last modified	Size	Storage class
☐	local-sample.txt	txt	February 6, 2026, 22:54:59 (UTC-05:00)	3.7 KB	Standard
☐	sample.txt	txt	February 7, 2026, 18:11:48 (UTC-05:00)	159.1 KB	Standard

ECR

Three private ECR repositories created for the splitter, mapper, and reducer services.

Private repositories (3)

Refresh

View push commands

Delete

Actions

Create repository

☐	Repository name	URI	Created at	Tag immutability	Encryption type
☐	mapreduce-mapper	09452269173 0.dkr.ecr.us-east-1.amazonaws.com/mapreduce-mapper	06 February 2026, 23:16:09 (UTC-05)	Mutable	AES-256
☐	mapreduce-reducer	09452269173 0.dkr.ecr.us-east-1.amazonaws.com/mapreduce-reducer	06 February 2026, 23:16:26 (UTC-05)	Mutable	AES-256
☐	mapreduce-splitter	09452269173 0.dkr.ecr.us-east-1.amazonaws.com/mapreduce-splitter	06 February 2026, 23:15:46 (UTC-05)	Mutable	AES-256

ECS tasks running

All 5 Fargate tasks running : 1 splitter, 3 mappers, and 1 reducer.

ARN
arn:aws:ecs:us-east-1:094522691:cluster/mapreduce-cluster

Status
Active

CloudWatch monitoring
Default

Registered container instances
-

Services

Draining
-

Active
-

Tasks

Pending
-

Running
5

Services

Tasks

Infrastructure

Metrics

Scheduled tasks

Configuration

Event history

Tags

Tasks (5)

Last updated
7 February 2026, 18:32 (UTC-5:00)

Manage tags

Stop

Run new task

Filter tasks by property or value

Filter desired status
Any desired status

Filter launch type
Any launch type

< 1 >

☐	Task	Last status	Desired status	Task definition	Health status	Created at
☐	3cf6ce151d224ebfb213a...	Running	Running	mapreduce-mapper:3	Unknown	58 minutes ago
☐	463bb16a79704b44abf9f...	Running	Running	mapreduce-mapper:3	Unknown	15 minutes ago
☐	47fe30ed6f3740f5838aa...	Running	Running	mapreduce-reducer:2	Unknown	57 minutes ago
☐	965f1308003b481caa1e7...	Running	Running	mapreduce-mapper:3	Unknown	13 minutes ago
☐	d4c90e03a87b4afc94c32...	Running	Running	mapreduce-splitter:2	Unknown	1 hour ago

Results with 1 mapper tasks


```

(base) srimansiramesh@Srimansis-Laptop Assignment4 % go run ./orchestrator s3://local-map-reducer/input/sample.txt

=====
MapReduce Word Count Orchestrator
=====
Input: s3://local-map-reducer/input/sample.txt
Chunks: 3
Mappers: 1

[1/3] Splitting input file...
  ✓ Split into 3 chunks in 221.262167ms
    Chunk 0: s3://local-map-reducer/chunks/chunk_1770505954_0.txt
    Chunk 1: s3://local-map-reducer/chunks/chunk_1770505954_1.txt
    Chunk 2: s3://local-map-reducer/chunks/chunk_1770505954_2.txt

[2/3] Mapping chunks in parallel...
  ✓ All 3 mappers completed in 445.197084ms
    Mapper 0: 2203 unique / 9859 total words → s3://local-map-reducer/mapped/map_result_1770505954660711585.json
    Mapper 1: 2373 unique / 9862 total words → s3://local-map-reducer/mapped/map_result_1770505954657889074.json
    Mapper 2: 2275 unique / 9858 total words → s3://local-map-reducer/mapped/map_result_1770505954697321970.json

[3/3] Reducing mapper outputs...
  ✓ Reduce completed in 189.307ms
    Final: 4797 unique words, 29579 total words
    Output: s3://local-map-reducer/results/final_wordcount_1770505954.json

=====
Performance Summary
=====
Split: 221.262167ms (25.9%)
Map: 445.197084ms (52.0%)
Reduce: 189.307ms (22.1%)
Total: 855.766251ms
=====

Timings saved to timings.json

```

Result with 3 mapper tasks

```

(base) srimansiramesh@Srimansis-Laptop Assignment4 % go run ./orchestrator s3://local-map-reducer/input/sample.txt

=====
MapReduce Word Count Orchestrator
=====
Input: s3://local-map-reducer/input/sample.txt
Chunks: 3
Mappers: 3

[1/3] Splitting input file...
  ✓ Split into 3 chunks in 267.533958ms
    Chunk 0: s3://local-map-reducer/chunks/chunk_1770506633_0.txt
    Chunk 1: s3://local-map-reducer/chunks/chunk_1770506633_1.txt
    Chunk 2: s3://local-map-reducer/chunks/chunk_1770506633_2.txt

[2/3] Mapping chunks in parallel...
  ✓ All 3 mappers completed in 224.311083ms
    Mapper 0: 2203 unique / 9859 total words → s3://local-map-reducer/mapped/map_result_1770506633645258496.json
    Mapper 1: 2373 unique / 9862 total words → s3://local-map-reducer/mapped/map_result_1770506633677352573.json
    Mapper 2: 2275 unique / 9858 total words → s3://local-map-reducer/mapped/map_result_1770506633700812875.json

[3/3] Reducing mapper outputs...
  ✓ Reduce completed in 191.058583ms
    Final: 4797 unique words, 29579 total words
    Output: s3://local-map-reducer/results/final_wordcount_1770506633.json

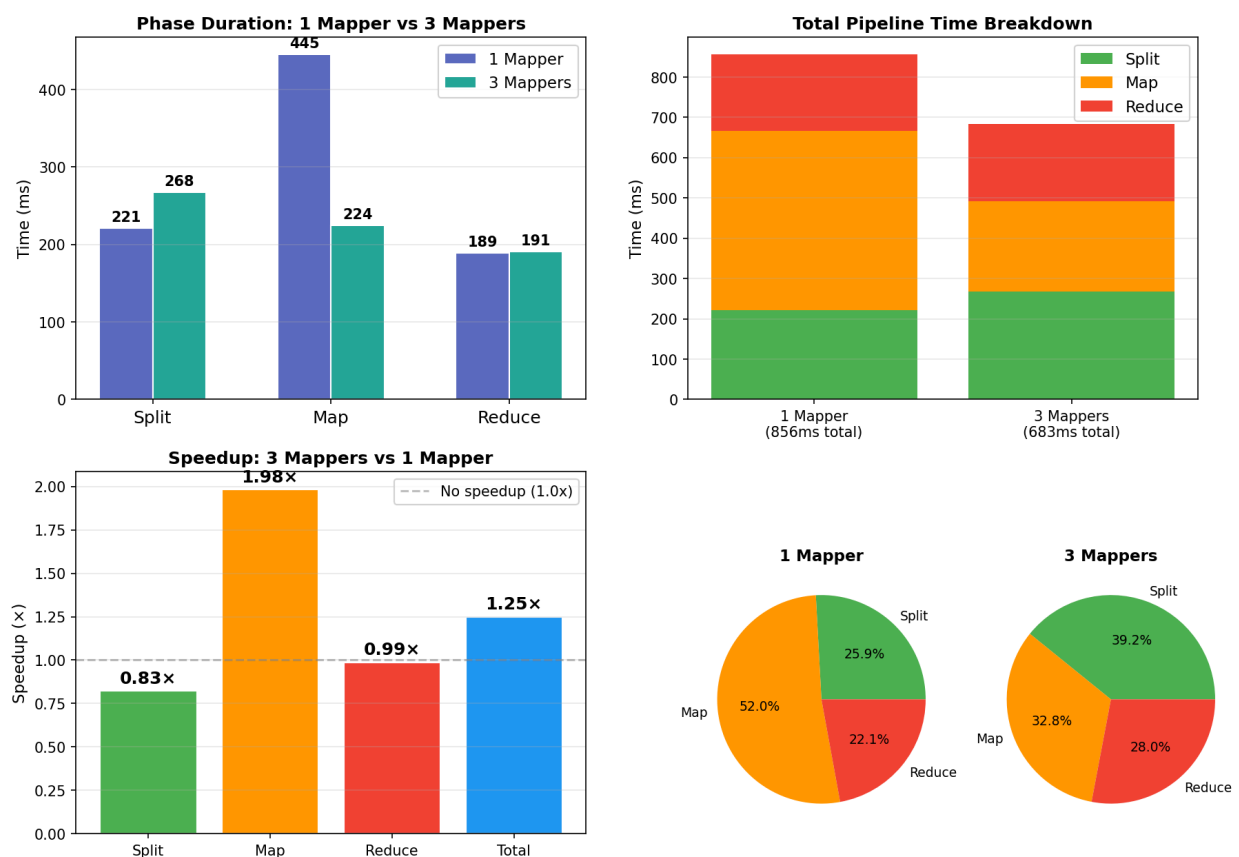
=====
Performance Summary
=====
Split: 267.533958ms (39.2%)
Map: 224.311083ms (32.8%)
Reduce: 191.058583ms (28.0%)
Total: 682.903624ms
=====

```

Performance Comparison:

Phase	1 Mapper	3 Mappers	Difference
Split	221ms	268ms	~same
Map	445ms	224ms	~2x faster
Reduce	189ms	191ms	~same
Total	856ms	683ms	20% faster

MapReduce Word Count — Performance Results
(29,579 words | 4,797 unique | 3 chunks)



Running the same 29,579-word file through the pipeline with 1 mapper versus 3 mappers reveals a clear parallelism benefit. The Map phase dropped from 445ms to 224ms, a nearly 2x speedup because each of the three chunks was

processed concurrently on a separate ECS Fargate task rather than sequentially on a single machine. The Split and Reduce phases remained unchanged at roughly 220ms and 190ms respectively, which is expected since they are inherently sequential operations. Overall, total pipeline time decreased from 856ms to 683ms, a 20% improvement.

Verification of results:

Direct single-pass word count matches MapReduce output exactly .4,797 unique words, 29,579 total, with all top-10 word counts identical.

```
((base) srimansiramesh@Srimansis-Laptop Assignment4 % python scripts/verify.py scripts/sample.txt results/result.json

Direct count: 4797 unique words, 29579 total
MapReduce count: 4797 unique words, 29579 total

RESULT: PERFECT MATCH! MapReduce output is correct.

Top 10 words:
✓ the          direct= 993 mapreduce= 993
✓ and          direct= 862 mapreduce= 862
✓ to           direct= 683 mapreduce= 683
✓ of           direct= 610 mapreduce= 610
✓ i            direct= 547 mapreduce= 547
✓ you          direct= 522 mapreduce= 522
✓ my           direct= 502 mapreduce= 502
✓ a            direct= 497 mapreduce= 497
✓ it           direct= 415 mapreduce= 415
✓ in           direct= 384 mapreduce= 384
```